

UNIFY LEARN

Product Documentation

Guided 12-week paths, XP, streaks and notes that fit your courses.

Version 1.11.0 · 4 October 2026

Mobile-first learning platform for the Faculty of Engineering, Lagos State University.

Covers: current features · roles & journeys · notifications · sessions & security ·
design system · architecture · pilot readiness · multi-tenant roadmap.

Contents

- 1. 1 · Product overview
- 2. 2 · Roles & access matrix
- 3. 3 · Student journey
- 4. 4 · Author journey (lecturers & collaborators)
- 5. 5 · Admin journey
- 6. 6 · Notifications (in-app · email · push)
- 7. 7 · Sessions, security & XP economy
- 8. 8 · Reader: story notes & quizzes
- 9. 9 · Design system (Classic / Story, dark mode)
- 10. 10 · Versioning scheme
- 11. 11 · Architecture & data model
- 12. 12 · Pilot readiness: the 500-user verdict
- 13. 13 · Multi-tenant production plan
- 14. 14 · Roadmap ("later")
- 15. 15 · Appendix: environments & operations
- 16. 16 · v1.7–v1.8 additions
- 17. 17 · v1.8.1–v1.11 bug-fix batch + new features

1 · Product overview

Unify Learn is a mobile-first web application that turns lecture material into guided, interactive weekly notes for engineering students. Authors publish versioned topics; students read, attempt quizzes, earn XP and keep streaks; admins run the whole platform from an oversight dashboard.

- **Frontend:** Vite + React + TypeScript, served on Vercel (`unify-virid.vercel.app`).
- **Backend:** Express + TypeScript + Prisma on Render (`unify-api-z4zm`).
- **Data & auth:** Supabase Postgres + Supabase Auth.
- **Email:** Brevo (SMTP for auth mail, API for note notifications).
- **Push:** Web Push (VAPID) to browsers, third notify leg.
- **Content seed:** 158 mined courses, 90 legacy weeks converted to topic-level notes.

2 · Roles & access matrix

Capability	Student	Lecturer	Collaborator	Admin (role)
Learn paths (My Courses, Explore, reader)	Yes (enrolled only)	No	No	No (reads via All content)
XP, streaks, quizzes recorded	Yes	No	No	No
Studio authoring + preview	No	Yes	Yes	Yes
Level-scoped note browser	—	Teaching level	Contributing level	All content
Admin panel (users, courses, analytics)	No	No	No	Yes (role or legacy flag)
Role assignment	Immutable after onboarding; changed only by an admin (recipient is notified).			

The bootstrap `unify.admin` account is flag-based (`collaborator` + `is_admin`), so the flag permanently counts as admin — the main account can never be locked out. Other admins inherit full rights via the `admin` role.

3 · Student journey

1. **Auth:** email sign-up (verification mail), sign-in, Google (coming soon), forgot-password reset, resend verification. Remember-me is opt-in.
2. **Onboarding (8 steps):** role → name → university → faculty → department → level → courses (own level + active semester) → graduation target.
3. **Dashboard:** greeting, daily line, XP / streak / target / courses, Continue-Learning resume card (exact course-week-topic), quiz stats, enrolled courses.
4. **My Courses:** enrolled courses with week counts. **Explore:** enroll-only catalog for own level + active semester (single bulk call).
5. **Reader:** story hero, segmented chapter bar, topics, recalls, quizzes, read-aloud, resume tracking.
6. **Notifications:** bell badge + list page; unread auto-opens on fresh entry. Email + push per profile prefs.
7. **Profile:** inline edit, email change, notify prefs, appearance (Classic/Story, light/dark, push toggle), version footer, guarded logout.

4 · Author journey

1. **Onboarding (lecturer 6 steps / collaborator 3):** both pick the level they teach / contribute to; courses selected enroll as *teaching*.
2. **Studio:** three methods — AI generate (Gemini, rate-limited), manual visual builder (no raw JSON), external-AI format pack.
3. **Versioning:** every publish stacks v1, v2, v3... per topic; authors delete own versions, admins any. Reader shows version badges + picker.
4. **Dashboard:** Notes / Courses / Students / Done stats, own published list with preview links, level-scoped Browse entry.
5. **Contributing level:** saved at onboarding, changeable later in Profile; scopes the Browse tree.

5 · Admin journey

- **Oversight dashboard:** platform totals (users, courses, weeks, topics), latest signups, latest published notes. Main admin (role) gets the panel link; flag admins get view-only notes.
- **Admin panel modules:** Analytics (roles donut, 14-day signups/notes/XP charts), AI models, Universities, Courses per level (search), Academic session + promote-all, Announce, Users & roles (search, invite, role/level edit, self-lock).
- **All content browser:** every course → weeks → topic/version chips with read-only preview links.
- **Academic control:** active-semester toggle (students only ever see that semester), one-click promotion (500 Level graduates, enrollments reset).
- **Safety:** own role locked (UI + 403 server-side), self-demotion blocked, every destructive/broadcast action uses the designed confirm sheet.

6 · Notifications (in-app · email · push)

Event	In-app bell	Email (Brevo)	Push
New note published	Enrolled students	Confirmed + opted-in takers	Subscribed takers
Admin announcement	All users	—	All subscribed
Welcome (first onboarding)	New user	— (auth mail covers it)	—
Role promoted / demoted	Recipient	—	Recipient

Auth mail (signup verify, invite, password reset) uses three branded Supabase templates; everything else stays on Supabase defaults. Push needs VAPID keys on Render and per-device opt-in from Profile → Appearance.

7 · Sessions, security & XP economy

- **Independent per-tab sessions** (sessionStorage); Remember-me restores tokens across restarts (localStorage, opt-in).
- **30-minute sliding inactivity TTL** persisted across app close; expiry signs out with an explanatory message. Applies to remembered sessions too.
- **XP**: +10 per completed topic, students only (server-enforced; staff reads record nothing). Quizzes record stats, award no XP. Streaks count consecutive UTC days with XP events.
- **Guards**: enrolled-only learning (fail-open on flakes), role gates on every route, admin-only APIs via `requireAdminUser`, no 4xx retries, rate limits, log-only mode without keys.

8 · Reader: story notes & quizzes

- Dark story hero (eyebrow, serif title, outcome box, chips incl. XP earned), segmented chapter bar (done/current, tap-to-jump), scroll-reveal story beats, Back/Next auto-scroll, back-to-top landing on the topic head, print stylesheet.
- Tap-to-reveal recall deck with Got-it / Review-again self-grading and progress; key phrases highlighted via `<mark>`; designed fill-in-the-gap fields; read-aloud mode (device voice, per-part progress).
- End-of-week exam card: progress dots, instant verdicts with author feedback, topic-ref review chips, mascot pass/fail score screen with exact retry list.
- XP bursts on completion (+10 XP) and week-complete celebration; reduced-motion respected throughout.

9 · Design system

- **Classic (default)**: the previous look — white surfaces, Nunito, emerald.
- **Story (opt-in)**: the MEE352 foundation — warm paper, Playfair Display + DM Sans, editorial hero. Adopted from the original `mee352-week2` source.
- **Dark mode** re-skins either design; persisted, no flash on return. Toggle lives in Profile → Appearance with the design switch and push toggle.
- Confirmations everywhere use the designed bottom-sheet modal — zero browser popups.

10 · Versioning scheme

Semantic versioning `MAJOR.MINOR.PATCH`, shown in Profile (currently **v1.11.0**):

- **MAJOR** — platform generation (1 = current app).

- **MINOR** — feature batch (new screens, flows, capabilities).
- **PATCH** — bugfix/hotfix push with no new surface.

11 · Architecture & data model

Layer	Tech	Notes
Web	Vite 5 · React 18 · TS 5 · Router 6 · MathJax	Static on Vercel; offline worker (shell v6, push-enabled, content cache)
API	Express 4 · TS · Prisma (DDL only)	Render; <code>prisma migrate deploy</code> on every build
Data/Auth	Supabase Postgres + Auth	Service-role reads via API; pooler :6543, direct :5432 for migrate
Mail/Push/AI	Brevo · Web Push (VAPID) · Gemini 3.6-flash	All log-and-skip without keys

Core tables

`profiles` (role, level, is_admin, prefs) · `enrollments` (taking/teaching) · `courses` + `course_levels` · `weeks` (shells) · `topic_notes` (versioned content) · `topic_progress` · `xp_events` · `quiz_attempts` · `resume_state` · `notifications` · `push_subscriptions` · `app_settings` (semester) · `ai_models` · `universities`

API map (v1)

Auth/profile: `/me` · `PUT /me` · `/onboarding` · `/settings` · `/stats` · `/resume` · `enrollments (+repair)` · `universities` · `courses (+weeks counts)` · `course weeks/week detail` · `progress` · `quiz/attempt` · `authored` · `author/stats` · `notes get/delete` · `notifications (+read)` · `push subscribe/unsubscribe` · `publish ×2` · `admin: stats, activity, trends, content, announce, users, promote, courses, unis, semester, models.`

12 · Pilot readiness: the 500-user verdict

Verdict: YES for a 500-tester pilot — with two paid switches and one code fix (done). 500 registered testers using the app normally will fit; 500 *simultaneous* users on the free tier will not.

Layer	Free-tier limit	500-tester pilot
Vercel (static frontend + CDN)	Generous bandwidth	Fine as-is.
Render free API (512 MB, shared CPU, sleeps)	~20–50 concurrent, 30–60 s cold starts	Upgrade to Starter (\$7/mo): no sleep, 1 GB RAM.
Supabase free (500 MB DB, 5 GB bandwidth, pooled conns)	Fine for hundreds of MAU	Fine as-is; keep the :6543 pooler. Pro (\$25) only if you outgrow it.
App N+1 (Explore fired ~160 requests/visit)	Would melt Render in a pilot	Fixed (v1.6.0): bulk week counts, one request per catalog.

Pilot runbook

1. Render Starter; confirm `/healthz` warm; set VAPID + Brevo + Gemini keys.
2. Seed + promote: run `seed.sql`, verify catalog, set active semester, create the admin.
3. Invite in waves (e.g. 5×100) with Supabase Auth logs + Render metrics open.
4. Watch: p95 latency, 5xx rate, Supabase connection count, Brevo daily quota (300/day free).
5. Success bar: signup → onboarding → first week → first quiz → Day-2 return for the cohort.

Do not pilot on Render free: cold starts alone will read as “the app is broken” to testers, and a class-wide signup surge will cascade into timeouts.

13 · Multi-tenant production plan (many universities)

Target: one deployment serving LASU + other universities, each isolated, each brandable, centrally billed. Recommended path — **single database, tenant-scoped rows** (not a DB per school):

1. **Tenant key:** add `tenants` table (subdomain, name, logo, colors, sender email); add `tenant_id` to profiles, courses, weeks, notes, enrollments, settings. Every query scopes by tenant (one middleware + RLS policies).
2. **Routing:** `lasu.unify.learn`, `unilag.unify.learn` → tenant resolver; keep path fallback (`?t=`) for testing.
3. **Auth:** email-domain rules per tenant (e.g. `@lasu.edu.ng` auto-joins LASU); invites carry tenant; one user can belong to two tenants (student here, lecturer there).
4. **Content:** shared starter catalog cloned per tenant on provisioning; each school curates its own weeks/notes/semesters.
5. **Admin:** tenant admins (current panel, scoped) + super-admins (cross-tenant analytics, provisioning, billing).
6. **Mail/push:** per-tenant sender identities (Brevo subaccounts/domains), per-tenant VAPID subjects.

7. **Infra:** Render Standard+ (or autoscaling host), Supabase Pro (backups, connection headroom, PITR), read-replica if analytics bite; per-tenant rate limits + quotas.
8. **Migration:** backfill all existing rows to the LASU tenant; dual-run; cut over per school. Estimated: 3–5 focused batches (schema → scoping → routing → admin → infra).

14 • Roadmap (“later”)

- Multi-tenant universities (plan above).
- Full offline packs (download a course, learn on zero data).
- Native-feel install prompts + iOS push parity notes.
- AI tutor over mastered topics; spaced-repetition scheduling from recall grades.
- Leaderboards, study groups, lecturer analytics per topic.
- Certificates + transcript export for departments.

15 • Appendix: environments & operations

Render (backend)

```
SUPABASE_URL • SUPABASE_PUBLISHABLE_KEY • SUPABASE_SECRET_KEY • DATABASE_URL (:6543) •  
DIRECT_URL (:5432) • CORS_ORIGIN • ADMIN_EMAILS • GEMINI_API_KEY • BREVO_API_KEY • EMAIL_FROM •  
VAPID_PUBLIC_KEY • VAPID_PRIVATE_KEY • VAPID_SUBJECT
```

Vercel (frontend)

```
SUPABASE_URL • SUPABASE_PUBLISHABLE_KEY • API_URL • USE_BACKEND=1 • VITE_VAPID_PUBLIC_KEY
```

(optional override)

Supabase dashboard

Custom SMTP (Brevo SMTP key, verified sender, test mail) • URL config (`unify-vid.vercel.app/**`) •
three branded templates (Confirm signup, Invite, Reset password) • Auth logs for signup 500s.

Repos & deploys

Full app: `knight-dev01/Unify` (Render builds backend, migrates DB). Frontend mirror: `DIBLS/Unify`
(Vercel) synced by `scripts/Sync-Frontend.ps1`. Every push: tsc both sides → commit → push → sync →
Vercel auto-build; Render auto-build migrates.

16 · v1.7–v1.8 additions

Expiring share links (viral loop)

- Authors and admins mint `/s/:token` links per week (lifetimes 8/16/24h, default 24h); students consume, never mint (403 server-side).
- Recipients read the full week free — hero, topics, recalls, exam, read-aloud — then hit expiry and a join screen. Views counted; owner/admin can revoke.
- WhatsApp/Telegram unfurls via a serverless OG page (`/api/share/:token`) with title, topics, countdown and cover; edge-cached 5 minutes.

XP-gated perks

- Save/Print PDF unlocks at **300 XP** (30 completed topics); staff bypass. Gate truthfully re-checks (retry + refocus, never a fake 0).
- Framework in `lib/xp.ts` (`XP_GATES`) for future locks: longer share lifetimes, early content, themes.

Nigerian reading voice

- Read-aloud auto-picks an `en-NG` device voice when present (many Androids ship Google's English (Nigeria)), with a manual voice picker and per-device memory.
- Honest limit: the accent must exist in the OS/browser TTS engine — the app cannot invent one. Android users download it under Settings → Language → Text-to-speech.

Design toggle & fonts

- Profile → Appearance switches **Classic** (previous look, default) and **Story** (MEE editorial) across the whole app, headings included.
- Theme and design are per-device (localStorage), cross-tab synced, never synced across browsers.

Push permission prompt

- Designed bottom sheet ~3s after first entry (7-day snooze), plus a browser-denied guide (site settings → allow; iPhone Home-Screen note).

17 · v1.8.1–v1.11 bug-fix batch + new features

October 2026 testing found 14 confirmed bugs (7 critical, 7 important). All fixed, each verified with `tsc` both sides plus a production build. Full per-bug account in `docs/DEVELOPER-REPORT.md`.

Content pipeline hardened (v1.8.1–v1.8.5)

- Collaborator scoping server-side; app-wide MathJax renderer (KaTeX removed); real Storage diagrams; async chunked generation with retry; submit-at-end EOQ; course aliases (`ME 352` → `MEE 352`) with cross-semester search; latest-version-only lists; Replace/Add/Cancel on occupied weeks.

- EOQ short banks auto-repair to exactly 10 questions (8 MCQ + 2 FITB); definition/formula cards render through MathJax with overlap-proof layout; quiz Check buttons styled.

Lecture structure (v1.9.0)

- Weeks hold **Lecture 1/2/3**, each with topics numbered from 1 and its own chapter tabs; the reader switches lectures without reload (`?c=`), with cross-lecture Back/Next. Progress, versions, resume and bookmarks are lecture-keyed. Studio/NoteBuilder carry lecture pickers; the AI prompt emits lecture groups. EOQ stays one week bank after the final lecture.

Dashboard, speed, Explore crash (v1.9.3–v1.9.4)

- Dashboard courses are a 2-column card grid (title, lecturer, weeks, topics done) with one-tap elective removal; catalog carries lecturer names; idle-time week-list warmup; Explore search-hook crash fixed.

Lecturer classes + offline-first (v1.10.0)

- **My Classes** (`/classes`): recurring weekly timetable per taught course (day/time/venue) plus a live roster (topics, quizzes, last activity). Students read class times on course pages.
- Offline foundation: progress/quiz/bookmark writes bank locally and flush on reconnect; per-week **Save offline** into a version-proof content cache (SW v6); banner shows the banked count.

Google sign-in (v1.11.0)

- Continue with Google on both auth tabs (Supabase OAuth, PKCE); first-timers onboard with their Google name prefilled. Requires the provider + redirect URLs in the Supabase dashboard (see `SETUP.md`).